

```
!pip install gensim
```

```
import numpy as np
import gensim.downloader as api
import itertools
import random
```

```
print("Loading GloVe 100D model...")
model = api.load("glove-wiki-gigaword-100")
print("Model Loaded Successfully!")
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
Loading GloVe 100D model...
[=====] 100.0% 128.1/128.1MB downloaded
Model Loaded Successfully!
```

```
def association(w, A, B):
    return np.mean([model.similarity(w, a) for a in A]) - \
           np.mean([model.similarity(w, b) for b in B])
```

```
def test_statistic(X, Y, A, B):
    return sum(association(x, A, B) for x in X) - \
           sum(association(y, A, B) for y in Y)
```

```
def effect_size(X, Y, A, B):
    assoc_X = [association(x, A, B) for x in X]
    assoc_Y = [association(y, A, B) for y in Y]
    mean_diff = np.mean(assoc_X) - np.mean(assoc_Y)
    std_dev = np.std(assoc_X + assoc_Y, ddof=1)
    return mean_diff / std_dev
```

```
def permutation_test(X, Y, A, B, num_samples=1000):
    combined = X + Y
    size = len(X)
    observed = test_statistic(X, Y, A, B)
    count = 0
```

```
    for _ in range(num_samples):
        random.shuffle(combined)
        new_X = combined[:size]
        new_Y = combined[size:]
        new_stat = test_statistic(new_X, new_Y, A, B)
        if new_stat > observed:
            count += 1
```

```
    return count / num_samples
```

```
male_words = ["man", "male", "boy", "brother", "he", "him", "son"]
female_words = ["woman", "female", "girl", "sister", "she", "her", "daughter"]
```

```
career_words = ["executive", "management", "professional",
                "corporation", "salary", "office", "career"]
```

```
family_words = ["home", "parents", "children",
                "family", "cousins", "marriage", "wedding"]
```

```
print("\nComputing WEAT Gender-Career Bias...\n")
```

```
es = effect_size(male_words, female_words, career_words, family_words)
p_val = permutation_test(male_words, female_words, career_words, family_words)
```

```
print("----- RESULT -----")
print(f"WEAT Effect Size : {es:.4f}")
print(f"P-value : {p_val:.4f}")
```

```
if es > 0:
    print("Bias Direction : Career is more associated with MALE terms")
```



```
elif es < 0:
    print("Bias Direction : Career is more associated with FEMALE terms")
else:
    print("Bias Direction : No detectable bias")

print("-----")
```

Computing WEAT Gender-Career Bias...

```
----- RESULT -----
WEAT Effect Size : 0.8400
P-value          : 0.0630
Bias Direction   : Career is more associated with MALE terms
-----
```

